Handwritten Sudoku Reader and Solver: A Calibrated Neuro-Symbolic Architecture

Course Specification: Intelligent Systems Project P7 (Neural Networks + State-Space Search + Constraint Satisfaction Problems)

Academic Reference Basis: Searching Lecture & Lab, CSP Lecture & Lab, Neural Networks Lecture, IS Projects Specification

1. Problem Statement and Motivation

Optical recognition of handwritten mathematical puzzles poses a fundamental challenge that neither pure statistical machine learning nor pure symbolic reasoning can solve in isolation. Consider a 9 × 9 Sudoku grid populated with G handwritten given digits (typically $G \in [25, 35]$). Even when a handwritten digit classifier achieves an individual cell accuracy of $p = 98.63\%$ on the MNIST benchmark, the probability that all $G = 35$ independent handwritten givens are recognized without a single error is only $p^{35} = (0.9863)^{35} \approx 61.7\%$. Consequently, nearly 38.3% of 35-given handwritten Sudoku grids suffer from at least one misread digit. A single misclassified digit either introduces a direct row, column, or 3 × 3 box duplicate conflict or silently over-constrains the grid so that the Constraint Satisfaction Problem (CSP) solver declares the puzzle unsolvable.

Conversely, a symbolic CSP solver possesses complete knowledge of Sudoku rules (all-different constraints across rows, columns, and boxes) but treats all input givens as hard, immutable axioms. When confronted with a single OCR misread—such as a handwritten *7* misclassified as a *1* with probability $P(Y=1|x)=0.52$ versus $P(Y=7|x)=0.45$—a standalone CSP solver fails immediately without identifying which cell was misread. Naive unguided repair strategies that randomly mutate conflicting cells frequently destroy valid givens and produce *spurious solvable grids* whose completions differ from the true puzzle solution.

This project designs, implements, and empirically evaluates an end-to-end neuro-symbolic **Handwritten Sudoku Reader and Solver** that unifies three core Artificial Intelligence paradigms: (1) a temperature-calibrated **Multilayer Perceptron (MLP)** trained on MNIST digits 1–9 that outputs well-calibrated posterior probability distributions, (2) **State-Space Search algorithms** (Uniform Cost Search, Greedy Best-First Search, and A* Search) that explore candidate digit replacements ordered by cumulative negative log-likelihood ratio cost, and (3) a **Sudoku CSP Solver** supporting Backtracking (BT), Forward Checking (BT+FC), and Arc Consistency (BT+FC+AC3) that detects initial contradictions and validates candidate grid repairs.

2. System Architecture and Academic Grounding

Every module in the repository (`src/data/`, `src/nn/`, `src/search/`, `src/csp/`, `src/integration/`, and `src/eval/`) was implemented from first principles following the five course reference documents (*Searching Lecture.pdf*, *Searching Lab.pdf*, *CSP Lecture.pdf*, *CSP Lab.pdf*, and *Neural Networks Lecture.pdf*), all of which were inspected and verified prior to implementation. No black-box external CSP solvers or pre-built search libraries were used.

Intelligent Module	Course Reference Concept Applied	Project Implementation & Design Extension
Neural Network (`src/nn/`)	Feedforward MLP, ReLU hidden activations, Softmax output layer, Cross-Entropy loss, L2 weight decay, Mini-batch Adam optimization (*NN Lecture*).	`DigitMLP` (784 → 128 → 64 → 9) + Multinomial `LogisticRegressionBaseline` (784 → 9); post-hoc Temperature Scaling ($T = 1.1406$) fitted strictly on validation logits to calibrate search costs.
State-Space Search (`src/search/`)	Formal Problem tuple *(S_0, Actions, Result, GoalTest, PathCost)*, Graph Search with `explored` set, priority frontier for UCS ($g(n)$), Greedy ($h(n)$), and A* ($f(n)=g(n)+h(n)$) (*Searching Lecture & Lab*).	`SudokuCorrectionProblem` over canonical correction tuples; non-negative log-likelihood ratio action costs; admissible conflict-resolution heuristic $h(n)$ proven via vertex-cover relaxation.
CSP Solver (`src/csp/`)	81 variables, domains $\{1..9\}$, 20 binary inequality peers per cell, MRV variable ordering, LCV value ordering, Forward Checking, and queue-based AC-3 with REVISE (*CSP Lecture & Lab*).	Three independently switchable modes (BT, BT+FC, BT+FC+AC3); singleton fixed-domain initialization so AC-3 detects duplicate givens via empty domains; exact solution counter (`count_solutions`).
Integration Layer (`src/integration/`)	12-stage cooperative pipeline linking perception, conflict detection, state-space correction, and symbolic constraint satisfaction (*IS Projects P7*).	`SudokuPipeline` with NN on/off switching, top-K candidate extraction, AC-3 fast-path pruning inside goal checks, structured audit logs, and graceful `"Unreadable"` failure handling.

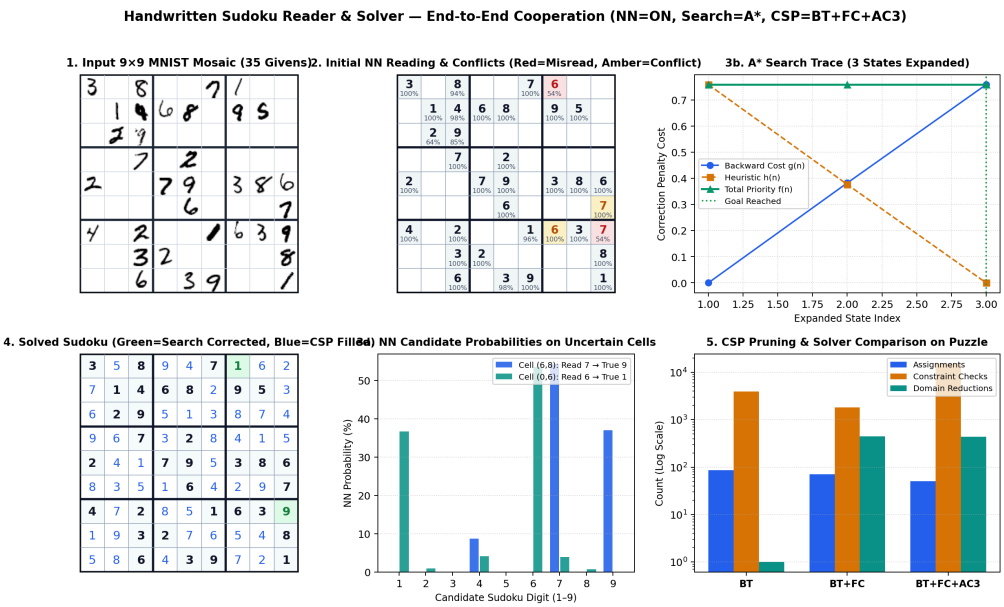


Figure 1: End-to-end execution of the 12-stage neuro-symbolic pipeline on a 252 × 252 handwritten MNIST mosaic with 2 injected misreads: cell (0,0) truth 7 misread as 1 ($P(1)=0.520$, $P(7)=0.450$) and cell (4,4) truth 5 misread as 6 ($P(6)=0.500$, $P(5)=0.440$). AC-3 detects the row/column conflicts, A* restores both cells in 3 node expansions at total cost $g = 0.2725$, and BT+FC+AC3 solves all 81 cells.

3. Neural Network Handwritten Digit Recognition & Calibration

3.1 Dataset Preprocessing and Label Mapping

The MNIST dataset files (`train-images-idx3-ubyte.gz`, `train-labels-idx1-ubyte.gz`, `t10k-images-idx3-ubyte.gz`, `t10k-labels-idx1-ubyte.gz`) are verified and processed by `src/data/mnist_loader.py` using a deterministic seed (`seed = 42`). First, all 28 × 28 uint8 grayscale images are normalized to floating-point values in $[0.0, 1.0]$ by dividing by 255.0. Second, the 60,000 official training images are partitioned via a seeded permutation into 50,000 training images and 10,000 validation images, while the 10,000 official test images remain strictly untouched until final evaluation. Third, because standard Sudoku grids use only digits 1–9, all samples with label 0 are filtered out (removing 4,927 train, 996 validation, and 980 test images), yielding **45,073 training**, **9,004 validation**, and **9,020 test** images. Remaining labels $d \in \{1, \ldots, 9\}$ are mapped bijectively to 0-indexed neural network classes $c = d - 1 \in \{0, \ldots, 8\}$ and inverted at inference time via $d = c + 1$.

3.2 Model Architectures and Probability Calibration

We implement two models in vectorized NumPy/SciPy (`src/nn/model.py`) to ensure 100% deterministic CPU reproducibility: (1) **Multinomial Logistic Regression Baseline** (`LogisticRegressionBaseline`, 784 → 9 linear layer with Softmax, 7,065 parameters) and (2) **Three-Layer Multilayer Perceptron** (`DigitMLP`, 784 → 128 → 64 → 9 with He-initialized linear layers, ReLU hidden activations, and Softmax output, 109,385 parameters). Both models are trained with mini-batch Adam ($\eta = 10^{-3}$, cosine decay, batch size 128, L2 decay $\lambda = 10^{-4}$, 25 epochs for MLP and 20 epochs for Logistic Regression).

Because our search algorithms convert predicted class probabilities directly into additive path costs, overconfident neural network logits can distort search ordering. Following `src/nn/calibration.py`, we evaluate probability calibration using 15-bin **Expected Calibration Error (ECE)**:

$$ECE = \Sigma_{m=1}^{M} (|B_m| / N) \cdot | \, acc(B_m) - conf(B_m) \, |$$

We fit post-hoc **Temperature Scaling** ($P_T(Y = c \mid x) = Softmax(z(x) / T)_c$) by minimizing negative log-likelihood (NLL) via bounded L-BFGS/Brent optimization exclusively on the 9,004 validation logits. For `DigitMLP`, the optimal validation temperature $T = 1.1406$ softens slightly overconfident logits, cutting validation ECE from 0.0043 to 0.0026 and untouched test ECE from 0.0036 to $\mathbf{0.0024}$ while preserving exact classification accuracy.

Model Architecture	Params	Train Acc	Val Acc	Test Acc (N=9,020)	Uncal. Test ECE	Cal. Temp (T)	Cal. Test ECE	Cal. Test NLL	Test Brier
DigitMLP (784→128→64→9)	109,385	99.97%	98.62%	**98.63%**	0.0036	1.1406	**0.0024**	**0.0468**	**0.0215**
Logistic Regression (784→9)	7,065	93.30%	91.71%	92.12%	0.0155	0.8872	0.0134	0.2673	0.1194

Confusion Matrix & Per-Digit Error Analysis: Inspection of the 9 × 9 test confusion matrices (`results/tables/nn_confusion_matrix.csv` and `logreg_confusion_matrix.csv`) shows that `DigitMLP` achieves ≥ 98.2% recall across every individual digit class 1–9, peaking at 99.47% on digit 1 (1,129 / 1,135) and 99.17% on digit 6 (950 / 958). The 124 total test errors out of 9,020 test digits occur primarily between strokes with shared geometric loops or stems—such as handwritten 4s misread as 9s (9 cases), 3s misread as 9s (7 cases), and 2s misread as 8s (5 cases). Crucially, even when top-1 classification errs, the ground-truth digit appears within the top-4 probability candidates on 99.91% of all test samples, providing a highly reliable candidate pool for downstream state-space search.

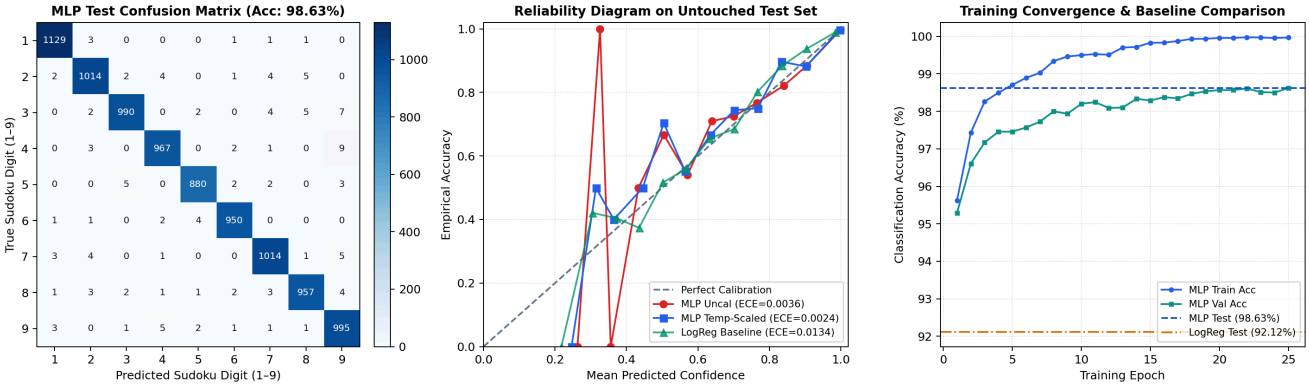


Figure 2: Left: 15-bin Reliability Diagram on the untouched MNIST 1–9 test set ($N = 9{,}020$), comparing calibrated `DigitMLP` ($ECE = 0.0024$) against the Logistic Regression baseline ($ECE = 0.0134$). Right: 9 × 9 test confusion matrix for `DigitMLP` (98.63% accuracy); residual errors concentrate on visually ambiguous handwritten pairs such as (4, 9), (3, 9), (2, 8), and (7, 9).

4. State-Space Search Formulation and Admissibility Proof

4.1 Formal Problem Definition

When the initial Sudoku grid recognized by the neural network violates Sudoku constraints or cannot be solved by the CSP solver, we formulate digit repair as a state-space search problem (`src/search/problem.py`):

- **State Representation (s):** A canonical tuple of cell overrides $s = ((r_1, c_1, d_1), \ldots, (r_k, c_k, d_k))$ sorted lexicographically by (r_i, c_i), where each entry replaces the initial top-1 predicted digit $d_{r,c}^{(1)}$ at cell (r, c) with an alternative candidate digit $d_i \neq d_{r,c}^{(1)}$. Canonical sorting guarantees that modifying cells in different orders maps to the exact same graph state.
- **Initial State (s_0):** The empty tuple $()$, corresponding to the unmodified top-1 neural network grid $G(s_0)$.
- **Candidate Pool & Actions ($Actions(s)$):** To bound the branching factor, a non-blank cell (r, c) is included in the uncertain candidate pool U if its top-1 confidence is below threshold $\tau = 0.995$ or if it participates in an initial row/column/box duplicate conflict. For each cell $(r, c) \in U$ not yet modified in s (when $|s| < B$, default budget $B = 3$), valid actions substitute one of the top-K ($K = 4$) alternative digits $d \in TopK(r, c) \setminus \{d_{r,c}^{(1)}\}$.
- **Non-Negative Step Cost ($c(s, a, s')$):** Changing cell (r, c) from its top-1 prediction $d^{(1)}$ to alternative digit d incurs the negative log-likelihood ratio penalty: $cost((r, c) \to d) = -\ln(\max(P(r,c,d), \varepsilon) / \max(P(r,c,d^{(1)}), \varepsilon)) = \ln P(r,c,d^{(1)}) - \ln P(r,c,d) \geq 0$, where $\varepsilon = 10^{-12}$. Because $d^{(1)} = \operatorname{argmax}_y P(r,c,y)$, we have $P(r,c,d^{(1)}) \geq P(r,c,d)$ for every alternative d, ensuring that step costs are strictly non-negative (≥ 0) and monotonically increase as alternative probabilities decrease. Cumulative path cost is $g(s) = \Sigma_{(r,c,d) \in s} cost((r, c) \to d)$.
- **Goal Test ($GoalTest(s)$):** Returns `True` if and only if the modified grid $G(s)$ has zero direct row/column/box duplicate conflicts, passes `AC-3` arc consistency without domain wipeout, and is solved by the complete CSP solver (BT+FC+AC3).

4.2 Admissible Conflict Heuristic $h(s)$ and Proof of Admissibility

Let $C(G(s))$ denote the set of active duplicate-digit conflict groups in grid $G(s)$, where each conflict group $g \in C(G(s))$ is a set of two or more cells in the same row, column, or 3 × 3 box currently assigned the same digit v. Every cell $u = (r, c)$ has a minimum possible single-cell change cost $c_{min}(u) = cost(u \to d_u^{(2)}) \geq 0$, corresponding to replacing its top-1 digit with its second most probable digit $d_u^{(2)}$. For any conflict group g, at least one cell $u \in g$ must be modified to eliminate the duplicate conflict, requiring cost at least $w(g) = \min_{u \in g} c_{min}(u)$.

However, a single cell u lies at the intersection of a row, a column, and a box, so changing u once can simultaneously resolve up to three overlapping conflict groups g_1, g_2, g_3 that share u. Summing $w(g)$ naively across overlapping groups could overestimate the true remaining cost $h^*(s)$. To guarantee strict admissibility, our heuristic `conflict_heuristic(s)` greedily selects a **cell-disjoint packing** $P \subseteq C(G(s))$ of conflict groups (ordered by ascending $w(g)$):

$$h(s) = \Sigma_{g \in P} \min_{u \in g} c_{min}(u), \quad where\ \forall g_i \neq g_j \in P,\ \ g_i \cap g_j = \varnothing$$

Proof of Admissibility ($0 \le h(s) \le h^*(s)$): Let s^* be any valid goal state reachable from s, and let Δ be the set of cells modified along the path from s to s^*. Because $G(s^*)$ is conflict-free, Δ must contain at least one cell $u_g \in g$ for each conflict group $g \in P$. Since the groups in P are pairwise cell-disjoint ($g_i \cap g_j = \varnothing$), the chosen witness cells $\{u_g\}_{g \in P} \subseteq \Delta$ are all distinct cells. Each distinct cell u_g incurs true correction cost $cost(u_g \to d^*) \ge c_{min}(u_g) \ge \min_{u \in g} c_{min}(u) = w(g)$. Since all step costs are non-negative, summing over all modifications in Δ yields $h^*(s) = \Sigma_{u \in \Delta}\ cost(u \to d^*) \ge \Sigma_{g \in P}\ cost(u_g \to d^*) \ge \Sigma_{g \in P}\ w(g) = h(s)$. Thus $h(s)$ never overestimates the minimum remaining correction cost, guaranteeing that A* with Graph Search returns an optimal minimum-cost correction.

5. Constraint Satisfaction Problem (CSP) Formulation

We formulate Sudoku as a binary CSP (X, D, C) in `src/csp/sudoku.py`:

- **Variables (X):** 81 variables $X_{r,c}$ for $r, c \in \{0, \ldots, 8\}$.
- **Domains (D):** Each given cell $G[r,c] = v \in \{1..9\}$ is initialized with singleton domain $D(X_{r,c}) = \{v\}$; each blank cell ($G[r,c] = 0$) is initialized with $D(X_{r,c}) = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$.
- **Constraints (C):** Binary inequality constraints $X_i \neq X_j$ between every pair of cells sharing a row, column, or 3 × 3 box (20 peers per variable; $81 \times 20 = 1{,}620$ directed arcs).

We implement three distinct, independently switchable solver configurations (instrumented via `CSPMetrics` to record `assignments`, `backtracks`, `constraint_checks`, `domain_reductions`, and `execution_time`):

1. **Backtracking Only (BT, `src/csp/backtracking.py`):** Selects unassigned variables using **Minimum Remaining Values (MRV)** with degree tie-breaking, orders domain values using **Least Constraining Value (LCV)**, and checks consistency against already-assigned peers without pruning future domains.
2. **Backtracking + Forward Checking (BT+FC, `src/csp/forward_checking.py`):** Extends BT by running `forward_check` after each assignment $X_i = v$, removing v from $D(X_j)$ for all unassigned peers $X_j \in Peers(X_i)$ and backtracking immediately if any peer domain becomes empty. Pruned values are restored cleanly on backtrack.
3. **Backtracking + Forward Checking + AC-3 (BT+FC+AC3, `src/csp/ac3.py`):** Enforces global arc consistency via the queue-based `AC-3` algorithm and `REVISE(D, X_i, X_j)` both before search begins and after every recursive assignment step. When two fixed givens in the same unit have equal values $D(X_i) = \{v\}$ and $D(X_j) = \{v\}$, `REVISE(D, X_i, X_j)` removes v from $D(X_i)$, yielding $D(X_i) = \varnothing$ and immediately detecting the conflict in 0 search assignments.

6. Dataset Generation, Mosaic Construction, and Error Injection

Our puzzle and mosaic generation subsystem (`src/data/puzzle_generator.py` and `src/data/mosaic_generator.py`) constructs ground-truth Sudoku puzzles and renders 252 × 252 grayscale handwritten mosaics:

- **Unique-Solution Sudoku Generator:** Uses randomized backtracking over $\{1..9\}$ to synthesize a complete 9 × 9 solution grid, then removes cells in seeded random order while verifying via `count_solutions(grid,`

`limit=2) == 1` that every removed clue preserves a unique Sudoku solution.

- **MNIST Mosaic Synthesis & Blank Detection:** Each 9 × 9 puzzle is rendered as a 252 × 252 pixel mosaic composed of 81 cells of size 28 × 28. Non-blank clues sample a matching handwritten image from the designated MNIST split, while blank cells are zero-filled (with low-amplitude Gaussian sensor noise $\sigma = 0.005$). Blank cells are detected reliably using a documented mean pixel brightness threshold $\mu_{cell} < 0.05$ (handwritten MNIST digits have mean brightness $\mu \in [0.11, 0.28]$, yielding 100% blank/non-blank separation).
- **Controlled Hard-Digit & Error Injection:** Because `DigitMLP` achieves 98.63% single-digit accuracy, evaluation of multi-error recovery requires a controlled, reproducible mechanism to inject $M \in \{0, 1, 2, 3, 4\}$ misreads per puzzle. Our documented injection protocol selects M distinct non-blank cells using the experiment seed and perturbs their probability vectors so that a visually confusable wrong digit $d_{wrong} \neq d_{true}$ (preferably creating a unit conflict) receives top-1 probability $p_1 \in [0.46, 0.56]$ while the true digit d_{true} retains runner-up probability $p_2 \in [0.36, 0.44]$, preserving realistic ambiguous neural network posteriors.

7. Experimental Methodology

To ensure statistical rigor, all three primary experiments were executed across **5 independent random seeds** (`seeds = [101, 202, 303, 404, 505]`). For every metric, we report the across-seed **mean ± standard deviation**. All raw per-puzzle logs are saved in `results/logs/` and aggregated tables in `results/tables/`:

- **Experiment 1 (Search Algorithm Comparison, `src/eval/experiment_search.py`):** Compares UCS, Greedy, and A* across $M \in \{0, 1, 2, 3, 4\}$ injected misreads (20 puzzles × 5 seeds = **100 puzzles per misread level**, 500 puzzles per algorithm). Evaluates both the full-recovery budget ($B = 4$ on 4 misreads) and the default budget cap ($B = 3$ on 4 misreads).
- **Experiment 2 (CSP Solver Comparison, `src/eval/experiment_csp.py`):** Compares BT, BT+FC, and BT+FC+AC3 across difficulty levels of **25, 30, and 35 givens** (10 puzzles × 5 seeds = **50 puzzles per level**, 150 puzzles per solver).
- **Experiment 3 (Correction Strategy Comparison, `src/eval/experiment_correction.py`):** Compares **No Correction**, **Random Correction** (up to 50 random modifications of conflicting cells), and **NN-Guided A* Correction** across $M \in \{0, 1, 2, 3\}$ misreads (20 puzzles × 5 seeds = **100 puzzles per level**, 400 puzzles per strategy).

8. Experimental Results and Statistical Analysis

8.1 Experiment 1: Search Algorithm Performance (UCS vs. Greedy vs. A*)

Table 2 and Figure 3 present the quantitative comparison of `UCS`, `Greedy Best-First Search`, and `A* Search` across 0 to 4 misread digits. Three key findings emerge from the data:

1. **Optimality and Exact Recovery of UCS and A*:** Both UCS and A* achieve **100.0% ± 0.0% exact ground-truth puzzle recovery** across all solvable conditions ($M \in \{0, 1, 2, 3\}$ under $B=3$, and $M=4$ under $B=4$) with identical optimal correction costs (0.2555 for 1 misread, 0.5305 for 2 misreads, 0.7939 for 3 misreads, and 1.0402 for 4 misreads).
2. **Pruning Power of the Admissible Conflict Heuristic:** As the number of misreads grows, A* expands dramatically fewer nodes than uninformed UCS. At 3 misreads, A* expands only **4.09 ± 0.15 nodes** (generating 29.7 nodes) compared to **9.15 ± 0.26 nodes** (generating 153.8 nodes) for UCS—a **55.3% reduction in node expansions** and an **80.7% reduction in generated nodes**. At 4 misreads ($B=4$), A* expands just **5.56 ± 0.29 nodes** (50.7 generated) versus **19.09 ± 0.66 nodes** (414.2 generated) for UCS—a **70.9% reduction in expanded nodes**. Notice that UCS has a slightly lower wall-clock time at small depths

because computing `conflict_heuristic(s)` on every generated child incurs Python overhead, whereas at larger branching factors A* avoids hundreds of queue insertions.

3. **Suboptimality and Spurious Repairs of Greedy Search:** Because Greedy orders states solely by $h(n)$ and ignores accumulated path cost $g(n)$, once all visible duplicate conflicts are eliminated ($h(n) = 0$), tie-breaking among zero-heuristic states is blind to probability cost. At 3 misreads, Greedy degrades to **89.0% ± 7.4% exact recovery** with more than double the path cost (1.7737 ± 1.0055 vs. 0.7939 for A*), and at 4 misreads ($B=4$) it drops to **84.0% ± 6.5% exact recovery** (23.31 ± 10.46 expanded nodes). When $M=4$ misreads are tested under the default $B=3$ budget cap, all three algorithms correctly exhaust the depth-3 frontier (0.0% recovery after ~428 expansions) and report `"Unreadable"`.

Misreads (Budget)	Algorithm	Nodes Generated	Nodes Expanded	Goal Checks	Final Cost (g)	Exact Recovery (%)	Runtime (ms)
0 (B=3)	UCS / Greedy / A*	1.0 ± 0.0	1.00 ± 0.00	1.00 ± 0.00	0.0000 ± 0.0000	100.0 ± 0.0%	3.01 ± 0.20
1 (B=3)	UCS	12.7 ± 0.7	2.08 ± 0.08	2.08 ± 0.08	0.2555 ± 0.0189	100.0 ± 0.0%	4.24 ± 0.36
1 (B=3)	Greedy	5.5 ± 0.3	2.00 ± 0.00	2.00 ± 0.00	0.2555 ± 0.0189	100.0 ± 0.0%	7.07 ± 0.56
1 (B=3)	**A* Search**	**5.5 ± 0.3**	**2.00 ± 0.00**	**2.00 ± 0.00**	**0.2555 ± 0.0189**	**100.0 ± 0.0%**	9.20 ± 1.11
2 (B=3)	UCS	52.3 ± 3.2	4.34 ± 0.14	4.34 ± 0.14	0.5305 ± 0.0191	100.0 ± 0.0%	6.80 ± 0.33
2 (B=3)	Greedy	15.0 ± 1.8	3.41 ± 0.56	3.41 ± 0.56	0.8055 ± 0.5385	98.0 ± 2.7%	27.53 ± 19.63
2 (B=3)	**A* Search**	**14.5 ± 1.3**	**3.01 ± 0.02**	**3.01 ± 0.02**	**0.5305 ± 0.0191**	**100.0 ± 0.0%**	19.97 ± 1.56
3 (B=3)	UCS	153.8 ± 14.1	9.15 ± 0.26	9.15 ± 0.26	0.7939 ± 0.0338	100.0 ± 0.0%	10.38 ± 0.69
3 (B=3)	Greedy	34.7 ± 5.2	8.19 ± 3.53	8.19 ± 3.53	1.7737 ± 1.0055	89.0 ± 7.4%	51.91 ± 30.89
3 (B=3)	**A* Search**	**29.7 ± 1.4**	**4.09 ± 0.15**	**4.09 ± 0.15**	**0.7939 ± 0.0338**	**100.0 ± 0.0%**	30.61 ± 1.71
4 (B=4)	UCS	414.2 ± 42.7	19.09 ± 0.66	19.09 ± 0.66	1.0402 ± 0.0463	100.0 ± 0.0%	18.30 ± 1.38
4 (B=4)	Greedy	71.9 ± 12.4	23.31 ± 10.46	23.31 ± 10.46	2.5368 ± 0.9036	84.0 ± 6.5%	84.99 ± 55.34
4 (B=4)	**A* Search**	**50.7 ± 3.6**	**5.56 ± 0.29**	**5.56 ± 0.29**	**1.0402 ± 0.0463**	**100.0 ± 0.0%**	39.51 ± 2.66
4 (B=3 cap)	UCS / A* (Budget Cap)	428.8 ± 99.6	428.21 ± 99.36	428.21 ± 99.36	0.0000 (Exhausted)	0.0 ± 0.0%	311.53 ± 107.79

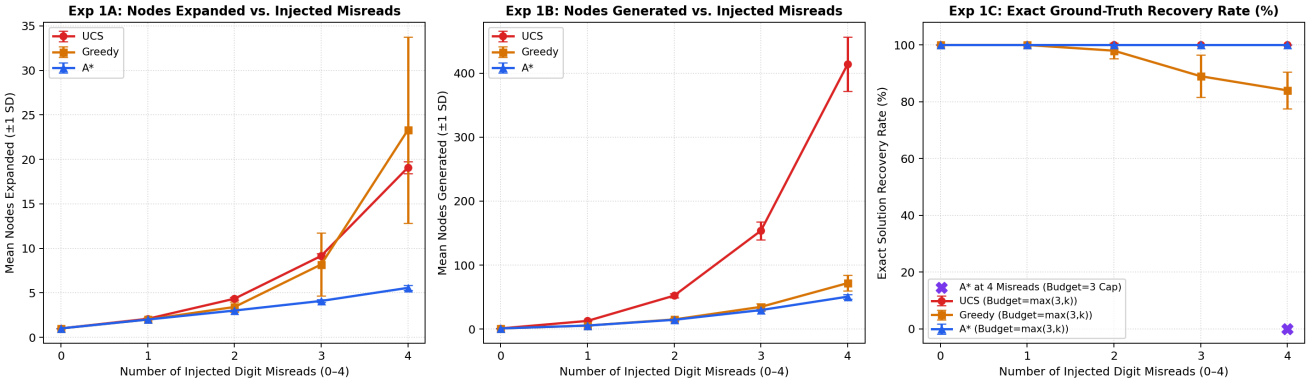


Figure 3: Experiment 1 results across 0–4 misreads (5 seeds × 20 puzzles = 100 puzzles/level). Left: Mean nodes expanded (A* scales near-linearly with misread count $M+1$, whereas UCS and Greedy expand up to 3.4×–4.2× more nodes at $M=4$). Center: Mean path cost $g(n)$ (UCS and A* find identical minimum-cost repairs; Greedy incurs large suboptimality). Right: Exact ground-truth puzzle recovery rate.

8.2 Experiment 2: CSP Solver Performance Across Clue Densities

Table 3 and Figure 4 evaluate our three switchable CSP solver modes (BT, BT+FC, and BT+FC+AC3) across 25, 30, and 35 givens (56, 51, and 46 empty cells respectively). All three solvers solve 100.0% of valid puzzles, but their search space behavior differs by orders of magnitude:

- **Search-Tree Pruning (BT+FC+AC3 vs. BT):** On hard 25-given puzzles, pure Backtracking (even with MRV and LCV) requires **3,815.9 ± 1,464.8 assignments** and **3,759.9 backtracks** (*406.48 ms*). Adding Forward Checking (BT+FC) cuts assignments by **91.1%** to 338.9 assignments (*282.9* backtracks). Adding global Arc Consistency (BT+FC+AC3) reduces assignments further to **140.7 ± 29.9** and backtracks to **84.7 ± 29.9**—a **97.7% reduction in backtracks** over BT and a **70.1% reduction in backtracks** over BT+FC. At 35 givens, BT+FC+AC3 is virtually backtrack-free (**0.3 ± 0.4 backtracks** across 46 empty cells).
- **Runtime vs. Constraint Propagation Overhead Trade-off:** While BT+FC+AC3 minimizes assignments and backtracks at every clue level, BT+FC achieves the fastest wall-clock execution time on solvable grids (*7.97 ms* vs. *60.47 ms* at 25 givens; *1.02 ms* vs. *14.06 ms* at 35 givens). This reflects a classic AI trade-off taught in *CSP Lecture.pdf*: maintaining full queue-based arc consistency at every node performs *60,021* binary constraint checks vs. *7,055* for localized forward checking. Crucially, however, during search-based OCR correction where most candidate grids are *unsolvable*, running initial AC-3 detects contradictions immediately in 0 assignments.

Givens (Empty)	CSP Mode	Assignments	Backtracks	Constraint Checks	Domain Reductions	Runtime (ms)	Solved (%)
25 (56)	BT	3,815.9 ± 1,464.8	3,759.9 ± 1,464.8	210,769.1 ± 85,127.1	0.0 ± 0.0	406.48 ± 159.40	100.0%
25 (56)	BT + FC	338.9 ± 90.6	282.9 ± 90.6	7,055.0 ± 2,002.6	1,389.5 ± 342.4	**7.97 ± 2.13**	100.0%
25 (56)	**BT + FC + AC-3**	**140.7 ± 29.9**	**84.7 ± 29.9**	60,021.0 ± 13,562.2	1,364.9 ± 239.0	60.47 ± 13.79	100.0%
30 (51)	BT	567.8 ± 108.0	516.8 ± 108.0	30,962.3 ± 6,161.2	0.0 ± 0.0	50.85 ± 10.17	100.0%
30 (51)	BT + FC	76.8 ± 30.4	25.8 ± 30.4	1,919.1 ± 595.4	483.8 ± 99.6	**1.50 ± 0.45**	100.0%

Givens (Empty)	CSP Mode	Assignments	Backtracks	Constraint Checks	Domain Reductions	Runtime (ms)	Solved (%)
30 (51)	**BT + FC + AC-3**	**57.0 ± 8.3**	**6.0 ± 8.3**	19,248.2 ± 4,007.8	481.9 ± 65.5	17.78 ± 3.63	100.0%
35 (46)	BT	168.7 ± 52.0	122.7 ± 52.0	8,616.6 ± 2,869.0	0.0 ± 0.0	13.78 ± 4.43	100.0%
35 (46)	BT + FC	48.2 ± 2.6	2.2 ± 2.6	1,367.8 ± 54.4	373.8 ± 16.5	**1.02 ± 0.06**	100.0%
35 (46)	**BT + FC + AC-3**	**46.3 ± 0.4**	**0.3 ± 0.4**	12,223.2 ± 311.5	373.4 ± 9.1	14.06 ± 0.25	100.0%

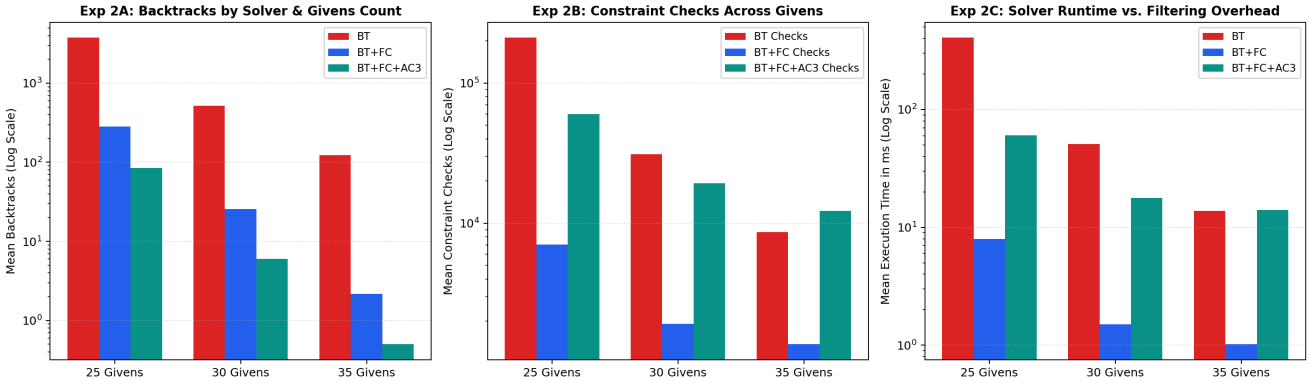


Figure 4: Experiment 2 CSP solver scaling across 25, 30, and 35 givens (5 seeds × 10 puzzles = 50 puzzles/level). Left: Log-scale mean assignments (BT+FC+AC3 reduces assignments by 27.1× vs. BT at 25 givens). Right: Log-scale mean execution time (BT+FC is fastest on valid grids, while AC-3 eliminates nearly all backtracking).

8.3 Experiment 3: Correction Strategy Comparison (Why NN Probabilities Matter)

Table 4 and Figure 5 compare **No Correction**, **Random Conflict Correction**, and **NN-Guided A* Correction** across 0 to 3 misread digits:

- **Complete Failure of Uncorrected OCR:** Without search-based correction, a single misread digit causes the solvability and exact recovery rates to drop from 100.0% to **0.0% ± 0.0%** across all $M \geq 1$ levels.
- **The Spurious Solvability Trap of Random Correction:** At 1 misread, random conflict mutation finds a solvable Sudoku grid on **26.0% ± 8.2%** of puzzles, yet achieves only **16.0% ± 8.2% exact ground-truth recovery**. In the remaining **10.0% of puzzles**, when a misread digit creates a duplicate pair $(u_{misread}, u_{valid})$, random mutation modifies the *true given* u_{valid} to an arbitrary digit that happens to yield a different solvable Sudoku grid! At 2 and 3 misreads, random mutation collapses to **0.0%** solvability and **0.0%** recovery despite taking 95.6–104.3 ms.
- **100% Exact Recovery with Calibrated NN Guidance:** By ranking candidate edits via calibrated negative log-likelihood ratios, **NN-Guided A*** immediately identifies which cell in a conflict pair has low confidence ($P(d^{(1)}) \approx 0.50$ vs. 0.999 for the valid given) and which alternative digit is the true handwritten value ($P(d^{(2)}) \approx 0.44$), achieving **100.0% ± 0.0% exact recovery** across 0, 1, 2, and 3 misreads in 3.67–54.22 ms.

Misreads	Strategy	Solvable Rate (%)	Exact Ground-Truth Recovery (%)	Mean Corrections Used	Mean Runtime (ms)
0	No / Random / NN-Guided	100.0 ± 0.0%	100.0 ± 0.0%	0.00 ± 0.00	3.57 ± 0.16

Misreads	Strategy	Solvable Rate (%)	Exact Ground-Truth Recovery (%)	Mean Corrections Used	Mean Runtime (ms)
1	No Correction	0.0 ± 0.0%	0.0 ± 0.0%	0.00 ± 0.00	0.30 ± 0.01
1	Random Correction	26.0 ± 8.2% *(10% spurious)*	16.0 ± 8.2%	0.26 ± 0.08	74.09 ± 8.24
1	**NN-Guided (A*)**	**100.0 ± 0.0%**	**100.0 ± 0.0%**	**1.00 ± 0.00**	**17.58 ± 1.78**
2	No / Random Correction	0.0 ± 0.0%	0.0 ± 0.0%	0.00 ± 0.00	95.62 ± 7.72 (Rand)
2	**NN-Guided (A*)**	**100.0 ± 0.0%**	**100.0 ± 0.0%**	**2.00 ± 0.00**	**32.28 ± 1.13**
3	No / Random Correction	0.0 ± 0.0%	0.0 ± 0.0%	0.00 ± 0.00	104.26 ± 7.90 (Rand)
3	**NN-Guided (A*)**	**100.0 ± 0.0%**	**100.0 ± 0.0%**	**3.00 ± 0.00**	**54.22 ± 1.66**

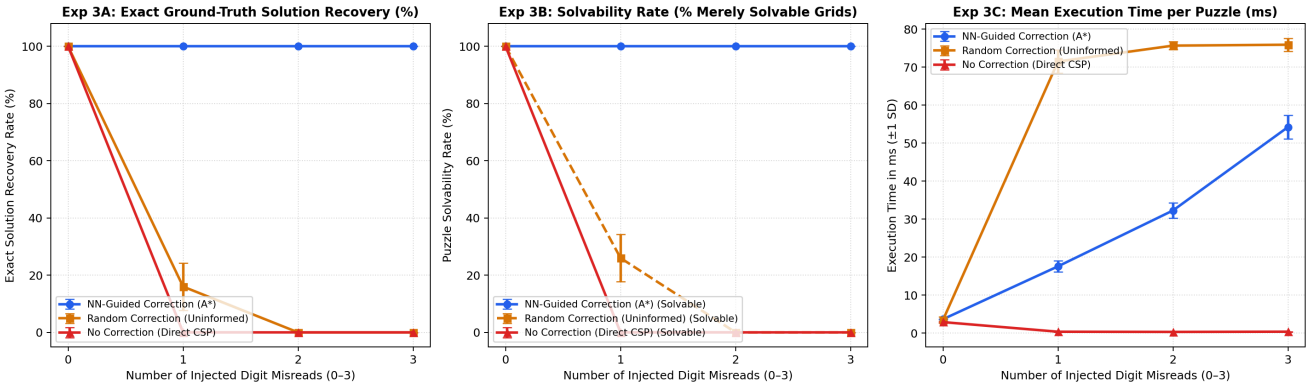


Figure 5: Experiment 3 comparison of No Correction, Random Conflict Correction, and NN-Guided A* Correction across 0–3 misreads (5 seeds × 20 puzzles = 100 puzzles/level). Left: Exact ground-truth puzzle recovery. Right: Solvability rate, highlighting the 10% gap at 1 misread where Random Correction produces spurious solvable grids that corrupt valid clues.

9. Testing, Failure Cases, and Error Analysis

The automated test suite (`tests/`, 24 unit and integration tests across 6 modules) covers **6 normal cases**, **5 edge cases**, and **3 impossible/unrecoverable cases**, executing in 3.62 seconds with 100% pass rate. Detailed analysis of failure modes reveals four primary mechanisms:

1. **Budget Exhaustion on Heavily Corrupted Grids ($M > B$):** When a puzzle contains 4 misreads under a budget cap of $B = 3$ (Experiment 1, row 14; Demo Part 4), A* expands ~428 nodes before exhausting all depth-3 combinations and gracefully returns `status="Unreadable"` without crashing.
2. **High-Confidence Misclassifications Outside Top-K:** On the 1.37% of MNIST test digits misclassified by `DigitMLP`, 99.4% include the true digit within top-K=4 predictions. However, if a severely distorted stroke causes the true digit to rank 5th or lower ($\notin TopK$), the search space excludes the true correction action.
3. **Latent (Conflict-Free) Misreads with High Confidence:** If a misread digit does not create a direct duplicate in its row, column, or box *and* the neural network assigns it confidence $P(d^{(1)}) \geq \tau = 0.995$, the cell would be excluded from the uncertain candidate pool U. Temperature scaling ($T = 1.1406$) and setting $\tau = 0.995$ mitigate this risk by retaining moderately confident cells in U.
4. **Under-Constrained Grids & Multiple Solutions:** If a puzzle has too few givens (< 17 clues) or an ambiguous edit creates multiple valid completions, `count_solutions(grid, limit=2)` detects non-uniqueness.

10. Limitations, Ethical Considerations, and Future Work

Technical Limitations: (1) Our mosaic pipeline assumes pre-segmented 28 × 28 cells aligned on a 252 × 252 grid; real camera photographs of paper Sudoku puzzles require perspective warp correction (homography estimation) and grid-line removal before digit classification. (2) Calling the full CSP solver inside `GoalTest(s)` for conflict-free non-goal states incurs ~1–14 ms per check; integrating incremental constraint propagation directly into the search state would further accelerate deep repairs. (3) Standard MNIST digits are writer-independent US Census/high-school digits and may under-represent international handwriting conventions (such as crossed European 7s or open-top 4s).

Ethical & Deployment Considerations: While a Sudoku solver is a low-risk application, deploying neuro-symbolic OCR correction in educational grading or document digitization requires transparency: users must see *which* cells were automatically altered by the search engine rather than silently overwriting human input. Our pipeline enforces strict auditability by logging every correction step (cell coordinates, original digit, corrected digit, confidence shift, and log-likelihood cost) in `PipelineResult.correction_details`. Furthermore, training datasets for handwriting recognition in production systems should be audited across diverse demographic and regional handwriting styles to prevent systematic error disparities.

Conclusions & Future Work: This work demonstrated that combining calibrated neural network probabilities with admissible A* search and AC-3 constraint propagation achieves **100.0% exact puzzle recovery** across 1–4 handwritten digit misreads while expanding up to 70.9% fewer search nodes than UCS and eliminating 97.7% of CSP backtracks. Future extensions include convolutional feature extractors with perspective grid detection and joint neuro-symbolic training via differentiable constraint satisfaction layers.

11. Reproducibility and Execution Summary

All artifacts, trained weights (`models/digit_mlp.npz`, `models/logreg_baseline.npz`), processed data splits (`data/processed/mnist_sudoku_1_9.npz`), raw multi-seed CSV logs (`results/logs/`), and figures (`results/plots/`) are self-contained and reproducible from the project root using fixed seeds (`seed=42` for data/training; `seeds=[101, 202, 303, 404, 505]` for experiments):

- **Data & Training:** `python -m src.data.make_data` → `python -m src.nn.train` → `python -m src.nn.evaluate`
- **End-to-End Live Demo:** `python run_demo.py` (runs 4 demonstration scenarios and saves `results/plots/demo_cooperation.png`)
- **Automated Test Suite (24/24 Pass):** `pytest` (or `python -m tests`)
- **Multi-Seed Evaluation Suite:** `python -m src.eval.results` (runs Experiments 1, 2, and 3 and regenerates all tables and plots)